

MANUAL INTERACTIVO · JAVASCRIPT · PARTE 2

Bucles for, Arreglos y tu primer sistema con datos

[]

Continuación de “De la lógica de PSeInt al código real en JavaScript”

`prompt()` · `alert()` · IDE libre + `index.html` · sin funciones

Lo que vas a aprender ahora

Ya sabés declarar variables, leer con `prompt()`, decidir con `if/switch` y repetir con `while`. Ahora vas a sumar más formas de repetir y a guardar y editar muchos datos juntos.

01

Entorno actual

Cualquier IDE + loader index.html

02

Bucles: for y do-while

Repetición controlada y validación garantizada

03

Arreglos (Arrays)

Equivalente a Dimension / Vectores

04

Manejo de arreglos

Agregar, modificar, eliminar y buscar por clave (DNI)

05

Proyecto: Control de Notas

Menú + arreglo + cálculos

06

Desafíos prácticos

3 ejercicios resueltos paso a paso

Por qué ahora cambiamos de método

No es que la Consola esté mal: a partir de acá los programas son más largos, y este método es más cómodo para iterar rápido.

Consola (lo que usábamos)

- ✓ Cómoda para ejercicios cortos de una sola pantalla.
- ✗ Incómoda para programas largos: cada cambio obliga a copiar y pegar todo de nuevo.

IDE + index.html (de acá en más)

- ✓ Cualquier editor sirve: Sublime Text, VS Code, o el que ya tengas.
- ✓ Un archivo index.html fijo carga el .js automáticamente.
- ✓ Ideal para programas con bucles, arreglos y menús largos como los que vienen.

Tu index.html: se crea una sola vez

```
index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Control de Notas</title>
</head>
<body>
  <script src="app.js"></script>
</body>
</html>
```

Flujo de trabajo

1

Creá index.html y app.js en la misma carpeta, con cualquier IDE.

2

Escribí y guardá todo tu código en app.js.

3

Abrí index.html con doble clic (o "Abrir con el navegador" desde tu IDE).

4

Para ver cambios: guardá app.js y recargá la pestaña (F5).

💡 *Mismo principio de siempre: editor separado del navegador. Ahora el loader queda fijo y ya no se edita más.*



MÓDULO 04

Bucles for y do-while

Repetir una cantidad exacta de veces, o hasta que se cumpla una condición

Anatomía del for

```
anatomia-for.js  
  
for (let i = 1; i <= 5; i++) {  
    alert("Vuelta número " + i);  
}
```

let i = 1

Inicialización — valor inicial del contador (se ejecuta una sola vez).

i <= 5

Condición — se evalúa antes de cada vuelta; mientras sea verdadera, el bucle sigue.

i++

Incremento — se ejecuta al final de cada vuelta (equivale a `i = i + 1`).

PSeInt

Para i <- 1 Hasta 5 Con
Paso 1 Hacer

FinPara

JavaScript

for (let i = 1; i <= 5; i++) {

}

En PSeInt las 3 partes (inicio, límite, paso) están separadas en palabras.

En JS están juntas en una sola línea, separadas por punto y coma.

Ejemplo · Repetir una pregunta N veces

```
repetir.js

// Pedimos cuántas veces se repite el saludo
let n = Number(prompt("¿Cuántas veces querés saludar?"));

// El for repite exactamente "n" veces, controlado por i
for (let i = 1; i <= n; i++) {
    alert("Hola número " + i + " de " + n);
}

alert("Listo, se repitió " + n + " veces.");
```

💡 Clave

A diferencia del while, en el for el contador se declara, se compara y se actualiza en la misma línea.

Es ideal cuando ya sabés cuántas veces se repite algo.

El bucle do...while (Repetir)

```
do-while.js
do {
    // este bloque se ejecuta primero,
    // sin importar la condición
} while (condicion);
```

do { }

Bloque que se ejecuta SIEMPRE al menos una vez, antes de preguntar nada.

while (condicion)

Se evalúa DESPUÉS de ejecutar el bloque. Si es verdadera, se repite.

;

A diferencia del while normal, el do-while termina con punto y coma.

PSeInt

JavaScript

Repetir

do {

Hasta Que condicion

} while (!condicion);

⚠ Cuidado con la lógica invertida:

PSeInt repite HASTA QUE la condición se cumpla (se detiene cuando es verdadera).

JS repite MIENTRAS la condición sea verdadera (se detiene cuando es falsa). Al traducir, la condición se niega.

while vs. do-while: la diferencia en la práctica

```
con-while.js

let numero = -1; // truco: arrancamos
                // con un valor inválido
while (numero <= 0) {
    numero = Number(prompt("Número > 0:"));
}

alert("Ingresaste: " + numero);
```

```
con-do-while.js

let numero;

do {
    numero = Number(prompt("Número > 0:"));
} while (numero <= 0);

alert("Ingresaste: " + numero);
```

💡 Con while necesitás inicializar la variable con un valor "trampa" para poder entrar al bucle la primera vez. Con do-while no hace falta: el bloque se ejecuta sí o sí antes de preguntar nada.



MÓDULO 05

Arreglos (Arrays)

Guardar muchos datos bajo un mismo nombre — el equivalente a Dimension y Vectores de PSeInt

De Dimension a Array

En PSeInt, para guardar varios valores relacionados usabas Dimension (un vector). En JS, la estructura equivalente es el array, y se declara con corchetes [].

PSeInt	JavaScript
Dimension notas[30] Como Entero	<code>let notas = [];</code>
(tamaño fijo, definido al declarar)	(tamaño dinámico, crece solo)
<code>notas[1] <- 8</code>	<code>notas[0] = 8;</code>



La diferencia más importante: PSeInt suele indexar desde 1. JavaScript SIEMPRE indexa desde 0. El primer elemento de cualquier array es arreglo[0], nunca arreglo[1].

Regla del índice 0 y .length

8

índice 0

6

índice 1

10

índice 2

4

índice 3

```
let notas = [8, 6, 10, 4];
```

`notas[0]` → primer elemento

`notas[3]` → último elemento

```
índice-length.js
```

```
let notas = [8, 6, 10, 4];
```

```
alert(notas[0]);           // 8    -> primer elemento
```

```
alert(notas[3]);           // 4    -> último elemento
```

```
alert(notas.length);       // 4    -> cantidad total de elementos
```

```
alert(notas[notas.length - 1]); // 4 -> forma segura de pedir "el último"
```

Cargar datos del usuario, uno por uno

carga-arreglo.js

```
let cantidad = Number(prompt("¿Cuántos números vas a cargar?"));

let numeros = []; // arreglo vacío al principio
let contador = 0; // cuenta cuántos ya cargamos

while (contador < cantidad) {
    let valor = Number(prompt("Núm " + (contador + 1) + ":"));
    numeros[contador] = valor; // se guarda en la posición "contador"
    contador = contador + 1;
}

alert("¡Cargaste " + numeros.length + " números!");
```

💡 Clave

Usamos while (no for) porque la carga depende de una condición de progreso (contador < cantidad) construida con prompt() en cada vuelta.

Es el mismo patrón de validación que ya conocés.

Mostrar todos los elementos con for

```
recorrer-arreglo.js

let numeros = [12, 45, 3, 78, 20];

for (let i = 0; i < numeros.length; i++) {
  alert("Posición " + i + ": " + numeros[i]);
}
```

💡 Patrón clave

```
for (let i = 0; i < arreglo.length; i++)
```

Empieza en 0, sigue mientras i sea menor a .length, y usa arreglo[i] para leer cada posición.

↻ Carga → while · Recorrido → for. Es la combinación que vas a usar en el proyecto integrador.

Agregar, modificar y eliminar elementos

JS permite modificar un arreglo después de creado. En PSeInt esto requería mover manualmente los datos con bucles; en JS el arreglo se reacomoda solo.

Método	Qué hace	Ejemplo
<code>push(valor)</code>	Agrega un elemento AL FINAL	<code>notas.push(8);</code>
<code>pop()</code>	Quita el ÚLTIMO elemento	<code>notas.pop();</code>
<code>unshift(valor)</code>	Agrega un elemento AL PRINCIPIO	<code>notas.unshift(10);</code>
<code>shift()</code>	Quita el PRIMER elemento	<code>notas.shift();</code>
<code>arreglo[i] = valor</code>	MODIFICA el elemento de la posición i	<code>notas[2] = 9;</code>
<code>splice(i, cant)</code>	QUITA "cant" elementos desde la posición i	<code>notas.splice(1, 2);</code>

⚠️ Estos métodos son propios de JS: no tienen un comando equivalente directo en PSeInt.

Ejemplo práctico · Lista de tareas

tareas.js

```
let tareas = [];  
  
let cantidad = Number(prompt("¿Cuántas tareas vas a cargar?"));  
let contador = 0;  
while (contador < cantidad) {  
    tareas.push(prompt("Tarea " + (contador + 1) + ":"));  
    contador = contador + 1;  
}  
alert("Cargaste " + tareas.length + " tareas.");  
  
// Agregar una tarea más al final  
tareas.push(prompt("¿Otra tarea para agregar?"));  
  
// Modificar la primera tarea  
tareas[0] = prompt("Reescribí la primera tarea:");  
  
// Eliminar la última tarea cargada  
tareas.pop();  
  
alert("Ahora tenés " + tareas.length + " tareas.");
```

💡 Clave

push() para agregar, arreglo[i] = valor para modificar, pop() para eliminar el último.

Todo sin usar function, igual que el resto del módulo.

Arreglos de objetos: clave y valor

Hasta ahora guardamos valores sueltos. También se puede guardar información completa por persona (DNI, nombre, nota) usando objetos dentro del arreglo — el equivalente a un Registro en PSeInt.

```
arreglo-objetos.js

let alumnos = [
  { dni: "40111222", nombre: "Ana", nota: 8 },
  { dni: "40333444", nombre: "Juan", nota: 6 }
];

alert(alumnos[0].nombre); // "Ana"
alert(alumnos[1].dni);    // "40333444"
alert(alumnos[0].nota);   // 8
```

Cómo leerlo

{ clave: valor, ... }

Cada elemento del arreglo es un objeto entre llaves, con pares clave: valor separados por coma.

alumnos[i].dni

Se accede a cada dato con un punto, igual que a un campo de Registro en PSeInt.

PSeInt

Registro persona ... FinRegistro

JavaScript

```
let alumnos = [ { dni:..., nombre:... }, ... ];
```

Buscar por DNI y modificar solo ese registro

buscar-modificar.js

```
let dniBuscado = prompt("Ingresa el DNI a modificar:");
let encontrado = false;

for (let i = 0; i < alumnos.length; i++) {
  if (alumnos[i].dni === dniBuscado) {
    let nombre = alumnos[i].nombre;
    let nuevaNota = Number(prompt("Nota para " + nombre + ":"));
    alumnos[i].nota = nuevaNota; // modificamos SOLO ese objeto
    encontrado = true;
    alert("Nota actualizada correctamente.");
    break; // ya lo encontramos, no hace falta seguir
  }
}

if (!encontrado) {
  alert("No se encontró ningún alumno con ese DNI.");
}
```

💡 Clave

El for recorre buscando una coincidencia exacta (===) con el DNI.

Al encontrarla, modificamos solo esa posición (`alumnos[i].nota = ...`) y usamos `break` para cortar la búsqueda.

Es el mismo patrón que usarías en PSeInt para buscar en un vector de registros.



PROYECTO

Sistema de Control de Notas

Un menú real con while, switch, un arreglo y for para calcular suma, promedio y nota máxima

Qué vamos a construir

Un único archivo app.js con un menú que se repite hasta elegir Salir, y un arreglo de notas que se carga una vez y se consulta las veces que haga falta.

while

Repetir el menú

switch

Dirigir las opciones

arreglo

Guardar las notas

for

Recorrer y calcular

alert()

Mostrar resultados

Paso 1

Menú + carga de notas

Estructura del menú y carga de notas en el arreglo, con validación 0–10.

Paso 2

Ver notas y promedio

Recorrido del arreglo con for y cálculo del promedio del curso.

Paso 3

Nota máxima

Recorrido con for e if para encontrar y mostrar la nota más alta.

Paso 1 · Estructura del menú y carga

app.js — Paso 1

```
let notas = []; // vive fuera del while: persiste entre opciones
let opcion = "0";
while (opcion !== "5") {
  opcion = prompt("1.Cargar 2.Ver 3.Promedio 4.Máxima 5.Salir");
  switch (opcion) {
    case "1":
      let cantidad = Number(prompt("¿Cuántas notas?"));
      notas = [];
      let contador = 0;
      while (contador < cantidad) {
        let nota = Number(prompt("Nota " + (contador + 1) + ":"));
        while (nota < 0 || nota > 10) {
          alert("Nota inválida (0 a 10).");
          nota = Number(prompt("Nota " + (contador + 1) + ":"));
        }
        notas[contador] = nota;
        contador = contador + 1;
      }
      alert("Cargadas: " + notas.length);
      break;
    case "5":
      alert("Saliendo...");
      break;
    default:
      alert("Opción inválida.");
  }
}
```

Paso 2 · Ver notas y promedio (con for)

app.js — Paso 2 (agregado al switch)

```
case "2":
    // Recorremos el arreglo con for para mostrar cada nota
    for (let i = 0; i < notas.length; i++) {
        alert("Alumno " + (i + 1) + ": " + notas[i]);
    }
    break;

case "3":
    let suma = 0;
    // El for acumula la suma de todas las notas
    for (let i = 0; i < notas.length; i++) {
        suma = suma + notas[i];
    }
    let promedio = suma / notas.length;
    alert("Promedio: " + promedio.toFixed(2));
    break;
```

💡 Acumulación

Una variable (suma) arranca en 0 y se actualiza en cada vuelta del for.

Es el mismo patrón que usabas en PSeInt para sumar dentro de un Para.

Paso 3 · Nota máxima (con for)

app.js — Paso 3 (agregado al switch)

```
case "4":
    let maxima = notas[0]; // suponemos que la primera es la más alta
    for (let i = 1; i < notas.length; i++) {
        if (notas[i] > maxima) {
            maxima = notas[i];
        }
    }
    alert("La nota más alta es: " + maxima);
    break;
```

✓ Resultado

El sistema queda completo: un arreglo cargado con while, recorrido con for, y un if que compara para quedarse con el valor más alto. Sin una sola función.

🔍 Para producción: antes de calcular promedio/máxima conviene validar `notas.length !== 0`, para evitar dividir por cero.



DESAFÍOS

3 ejercicios resueltos paso a paso

Mismo patrón: `prompt()` para cargar, `for` para procesar, `alert()` para mostrar

Desafío 1 · Sumar solo los múltiplos de 3

desafio1.js

```
let cantidad = Number(prompt("¿Cuántos números vas a cargar?"));
let numeros = [];
let contador = 0;

while (contador < cantidad) {
    numeros[contador] = Number(prompt("Núm " + (contador + 1) + ":"));
    contador = contador + 1;
}

let sumaMultiplos = 0;
for (let i = 0; i < numeros.length; i++) {
    if (numeros[i] % 3 === 0) {
        sumaMultiplos = sumaMultiplos + numeros[i];
    }
}

alert("La suma de los múltiplos de 3 es: " + sumaMultiplos);
```

Pasos resueltos

1. Cargar los números con while.
2. Recorrer el arreglo con for.
3. Un if filtra los múltiplos de 3 ($\% 3 === 0$).
4. Acumular con `sumaMultiplos = sumaMultiplos + numeros[i]`.

Desafío 2 · Encontrar el número más alto

desafio2.js

```
let cantidad = Number(prompt("¿Cuántos números vas a cargar?"));
let numeros = [];
let contador = 0;

while (contador < cantidad) {
    numeros[contador] = Number(prompt("Núm " + (contador + 1) + ":"));
    contador = contador + 1;
}

let mayor = numeros[0];
for (let i = 1; i < numeros.length; i++) {
    if (numeros[i] > mayor) {
        mayor = numeros[i];
    }
}

alert("El número más alto ingresado es: " + mayor);
```

Pasos resueltos

1. Cargar el arreglo (patrón ya visto).
2. Suponer que numeros[0] es el mayor.
3. Recorrer desde el índice 1 (el 0 ya se usó como punto de partida).
4. Actualizar mayor si se encuentra un valor más alto.

Desafío 3 · Contar positivos y negativos

desafio3.js

```
let cantidad = Number(prompt("¿Cuántos números vas a cargar?"));
let numeros = [];
let contador = 0;

while (contador < cantidad) {
    numeros[contador] = Number(prompt("Núm " + (contador + 1) + ":"));
    contador = contador + 1;
}

let positivos = 0;
let negativos = 0;

for (let i = 0; i < numeros.length; i++) {
    if (numeros[i] > 0) { positivos = positivos + 1; }
    if (numeros[i] < 0) { negativos = negativos + 1; }
}

alert("Positivos: " + positivos + " | Negativos: " + negativos);
```

Pasos resueltos

1. Cargar el arreglo (mismo patrón de siempre).
2. Dos contadores en 0: positivos y negativos.
3. Dentro del for, dos if independientes evalúan cada número.
4. Cada if que se cumple suma 1 a su propio contador.

¡A seguir programando!

Ya sabés repetir con for y do-while, guardar y editar datos con arreglos, y combinarlos con while, switch e if para construir programas reales — todo con prompt() y alert(), sin necesitar funciones.

🔄 **El patrón que no vas a olvidar:**

Cargar con while → Procesar con for → Mostrar con alert()